

CynLr

Cybernetics Laboratory

Design Overview Document

Line-Scan Scanner Pipeline

Evaluation 1 — Programming Fundamentals (C++)

Role:	Software Engineer — C++, Performance Coding & Architectures
Language:	C++17 Visual Studio 2017+ Windows / Linux
Stage:	Evaluation 1 — Objective Assessment
Covers:	Data Generation Block • Filter & Threshold Block

Table of Contents

1. Problem Summary & Scope	3
2. System Architecture	3
3. Design Patterns Employed	4
4. Block-by-Block Design	4
5. Optimisation Inventory	5
6. Inter-Block Communication	7
7. Scalability Analysis	8
8. Performance & Timing	8
9. Coding Standards	9
10. Unit Test Strategy	9
11. Project File Structure	10
12. Build & Execution Instructions	10
13. Design Questions Answered	11

1. Problem Summary & Scope

The system simulates a high-speed line-scan camera used in newspaper printing and textile inspection. It processes an effectively infinite 2-D pixel stream (m columns wide) at a fixed cycle rate T , applying a 9-tap Gaussian smoothing filter to each element, then binary-thresholding the result against a configurable value TV .

Constraint	Requirement
Throughput	Successive pixel output latency < 100 ns
Memory	Outstanding data in-flight $\leq m$ elements
Timing	Each pipeline block completes within its T window (≥ 500 ns)
Scalability	Adding future blocks (blocks 3/4) requires zero changes to existing code
Modularity	Each block is agnostic to internals of all other blocks

2. System Architecture

The solution adopts the **Pipeline architectural pattern** — a chain of processing stages connected by typed data channels — combined with the **Strategy pattern** for pluggable data sources.

```
main()
fork()

DataGenerationBlock → RingBuffer<PixelPair,256> → FilterThresholdBlock

[Block 3 – future, no existing changes needed]

join() / shutdown()
```

2.1 Class Hierarchy

- **IProcessBlock** (pure-abstract interface) — `configure()` → `start()` → `stop()` → `join()`
 - `DataGenerationBlock` — produces `PixelPairs` every T ns
 - `FilterThresholdBlock` — consumes `PixelPairs`, filters, thresholds
 - (Future blocks implement `IProcessBlock` — zero changes to existing code)

The **Pipeline** class owns a `std::vector<unique_ptr<IProcessBlock>>` and calls the lifecycle methods uniformly. Adding Block 3 means appending one new instance — nothing else.

3. Design Patterns Employed

Pattern	Application in this Codebase
Pipeline	Chain of processing stages; data flows in one direction through typed channels
Strategy	<code>DataSourceMode</code> selects <code>RandomGenerator</code> or <code>CsvFile</code> at runtime
RAII	Threads, file handles, ring-buffer slots managed by constructors/destructors
Observer	<code>OutputCallback</code> lambda decouples filter output from all consumers

Pattern	Application in this Codebase
Object Pool	Ring buffer raw-byte storage with placement new — zero heap alloc on hot path
Null Object	Default no-op OutputCallback avoids null checks in worker loop
Template Method	IProcessBlock defines lifecycle; concrete blocks fill configure/start/stop/join

4. Block-by-Block Design

4.1 DataGenerationBlock

Attribute	Detail
Role	Emulates line-scan camera; produces 2 uint8 pixels (PixelPair) per cycle T
Thread model	Dedicated std::thread; sleep_until(deadline – 50 μs) then busy-spin to deadline
Mode A	RandomGenerator — xorshift64 PRNG, 2 independent generators, ~3-4 cycles/call
Mode B	CsvFile — manual byte-scan parser, zero heap after initial file read
EOS sentinel	Sends PixelPair{eos=true} when CSV exhausted; consumer drains and exits cleanly
Output	push() or push_batch() into RingBuffer<PixelPair, 256>

4.2 FilterThresholdBlock

Algorithm for element K (9-tap 1-D convolution):

```
filtered(K) = sum[ raw[K+i] x w[i] ] for i in {-4, -3, -2, -1, 0, +1, +2, +3, +4}
output(K) = (filtered(K) >= TV) ? 1 : 0
```

Attribute	Detail
Filter type	9-tap symmetric Gaussian (spec weights, Q24 integer path)
Sliding window	Single 16-slot circular uint8 array — no std::deque, no heap
Memory overhead	16 bytes (win_[]) + 64-byte batch buffer — well within ≤ m budget
Pipeline latency	4-element look-ahead delay (unavoidable for non-causal filter)
Integer path	Q24 fixed-point: weights pre-scaled by 2 ²⁴ , sum stays in uint32
Threshold	Branchless arithmetic-shift trick: (threshold - filtered - 1) >> 31 & 1
Batch drain	pop_batch(64) — one acquire fence per 64 pairs vs one per pair
Border handling	Pre/post stream zero-padded (standard convolution convention)

4.3 Gaussian Filter Window Constants

Position	Double Weight	Q24 Weight	Notes
K-4	0.00025177	4	Far past

Position	Double Weight	Q24 Weight	Notes
K-3	0.008666992	145	
K-2	0.078025818	1308	
K-1	0.24130249	4045	Near past
K	0.343757629	5762	Centre (highest weight)
K+1	0.24130249	4045	Near future
K+2	0.078025818	1308	
K+3	0.008666992	145	
K+4	0.000125885	2	Far future (spec asymmetric tail)

5. Optimisation Inventory

Every optimisation listed below is annotated with the technique category, the specific change made, and the measured or theoretical benefit. All changes preserve correct functional output.

5.1 RingBuffer — Lock-Free SPSC Queue

Technique	Change	Benefit
Power-of-2 bitmask	(head+1) & MASK replaces (head+1) % Capacity	Eliminates integer division (8-40 cycles) → 1-cycle AND. Static_assert enforces power-of-2.
Placement new + raw byte pool	alignas(T) std::byte storage_[sizeof(T)*Capacity]	Zero default-constructor calls for unused slots. push() = placement new (single store for trivial T). No wasted copy-assign.
Cache-line isolation	alignas(64) PaddedAtomic head_ / tail_	Producer owns head_ cache line; consumer owns tail_ line. Eliminates false sharing — the #1 SPSC anti-pattern.
std::launder	slot() returns std::launder(reinterpret_cast(...))	Satisfies strict-aliasing rules for placement-new objects. Compiler cannot incorrectly optimise slot reads.
Batch push / pop	push_batch(items,n) / pop_batch(out,n)	ONE acquire + ONE release fence for N items. For BATCH_SIZE=64 pairs: 64x fewer memory barriers.
Relaxed load for own index	head_.load(relaxed) in push(); tail_.load(relaxed) in pop()	No cache coherency cost for the index this thread exclusively writes.

5.2 FilterThresholdBlock — Convolution & Threshold

Technique	Change	Benefit
Eliminate std::deque	Replaced lookahead_ deque + history_ array with single win_[16] circular array	Zero heap allocation on hot path. All 9 window elements in one cache line. No pointer chasing.
Q24 fixed-point integer filter	Weights pre-scaled x 2 ²⁴ . Convolution = uint8 x uint32 sum, right-shift by 24	Avoids FPU entirely. Integer multiply: 1-3 cycles vs 4-6 for FP64. Result stays in uint32.
Branchless threshold	(int32_t)(threshold_q24 - filtered_q24 - 1) >> 31 & 1	No conditional branch — eliminates branch misprediction on alternating output patterns.
Bitmask window wrap	winHead_ = (winHead_ + 1) & WMASK (WMASK = 15)	Single AND replaces modulo or branch-based wrap.
__builtin_expect hints	primed<9 and onOutput_!=nullptr paths marked unlikely	Branch predictor trained correctly; hot path (fully primed, no callback) has zero mispredicts.

Technique	Change	Benefit
PAUSE instruction	<code>__asm__ volatile('pause')</code> in spin-wait	Reduces power consumption and memory-order pressure during brief idle spins; signals hyper-thread.
Unrolled convolution	9 multiplies written as explicit sum expression	Enables compiler auto-vectorisation (SIMD) with <code>-O2</code> . Removes loop overhead.
Batch consume	<code>pop_batch(batch_, 64)</code> per iteration	64x amortisation of acquire fence cost vs element-by-element pop.

5.3 DataGenerationBlock — PRNG & CSV Parser

Technique	Change	Benefit
xorshift64 PRNG	Replaces <code>std::mt19937</code> (2.5 KB state, ~15 cyc/call)	State = 8 bytes, ~3-4 cycles/call. Two independent generators for <code>pixel1/pixel2</code> . Top byte extracted via <code>>> 56</code> .
Manual CSV parser	Byte-scan over <code>mmap'd/read</code> buffer; digit accumulation with multiply-add	~5-10x faster than <code>std::istream</code> + <code>std::stoi</code> . Zero heap allocation per token. No locale overhead.
Flat offset tracking	<code>csvOff_flat</code> index replaces (<code>csvRow</code> , <code>csvCol</code>) pair	Single increment + mask check vs two separate conditionals.
Batch push	Accumulate <code>BATCH_SIZE</code> pairs locally, then <code>push_batch()</code>	Matches filter block batch consumption. Reduces fence pairs by <code>BATCH_SIZE</code> .

6. Inter-Block Communication

Blocks communicate exclusively through a typed **RingBuffer<PixelPair, 256>** — a lock-free SPSC queue that requires no mutex, no condition variable, and no kernel involvement on the fast path.

Property	Detail
Lock-free SPSC	Single producer + single consumer → wait-free progress guaranteed by construction
Capacity = 256	Power-of-two → bitmask wrap. Holds >m outstanding pairs at any time.
PixelPair layout	<code>#pragma pack(1): pixel1(u8) + pixel2(u8) + eos(bool) = 3 bytes</code> . Fits in one cache line slot.
EOS sentinel	<code>eos=true</code> in the pair itself — no out-of-band signalling channel needed
Acquire/Release	Producer: release on head after write. Consumer: acquire on head before read. Zero UB, minimal coherency traffic.
Extensibility	Adding Block 3 → add <code>RingBuffer<OutputToken,256></code> <code>channel2</code> between blocks 2 and 3. Zero existing changes.

7. Scalability Analysis

7.1 Adding a Third Block

Exactly three steps — zero existing files modified:

- Create DefectClassifierBlock.h/.cpp implementing IProcessBlock
- Instantiate: `auto b3 = make_unique<DefectClassifierBlock>(cfg, channel2, cb);`
- Append: `pipeline.addBlock(std::move(b3));`

7.2 Scalability Table

Change Scenario	Files Modified	Existing Code Changed?
Add Block 3 (future evaluation)	New block .h/.cpp only	No — existing code untouched
Change filter kernel	FILTER_WINDOW / FILTER_Q24 constant	One constant line only
Switch data source at runtime	CLI flag --csv or config field	No
Change ring buffer capacity	PipelineConfig::RING_CAPACITY	One constant (must remain power-of-2)
Add CSV output dump	OutputCallback lambda in main.cpp	No
Replace PRNG algorithm	Xorshift64 struct in DGB header	No change to pipeline or filter

8. Performance & Timing

8.1 Measured / Theoretical Metrics

Metric	Measured / Estimated	Requirement	Status
Filter compute — Q24 integer path	~0.8 ns/element (9 int mults + shifts)	< 100 ns	PASS
Filter compute — FP64 path (old)	~3-6 ns/element (FPU, no SIMD)	< 100 ns	PASS
RingBuffer push (fast path)	< 5 ns (no contention, bitmask wrap)	—	—
RingBuffer batch-64 amortised	< 0.1 ns per element	—	—
False sharing (old, no padding)	50-200 ns stall per cache-line bounce	—	FIXED
xorshift64 vs mt19937	~3 cyc vs ~15 cyc per call	< T	PASS
Memory overhead per element	3 bytes (PixelPair) + 16 bytes win_[]	≤ m bytes	PASS

8.2 Timing Discipline — Sleep + Spin Hybrid

- **Phase 1:** sleep_until(deadline - 50 μ s) — OS scheduler, low CPU burn
- **Phase 2:** Busy-spin until steady_clock::now() $\geq$ deadline — sub- μ s precision
- **PAUSE instruction** in spin-wait — reduces power and signals hyper-thread

This hybrid avoids OS scheduler jitter (typical overshoot 50-200 μ s on Linux) while keeping CPU usage minimal for the bulk of the T interval.

9. Coding Standards

Standard	Approach
Language	C++17 (-std=c++17). No vendor extensions. constexpr, if constexpr, std::launder, std::byte.
Naming	CamelCase types, snake_case members/functions, UPPER_SNAKE constants and macros
Documentation	Doxygen @brief / @param / @return on every public API. Inline rationale for every optimisation.
Headers	#pragma once. Minimal transitive includes. Forward declarations preferred over full includes.
RAII	All resources (threads, file handles, ring-buffer slots) managed by constructors/destructors.
Error handling	configure() returns bool. Exceptions only for fatal CLI parse errors (startup only).
Warnings	Clean build with -Wall -Wextra -Wpedantic -O2. No suppressions.
Thread safety	Atomic flags for lifecycle. SPSC ring buffer avoids shared mutable state entirely.
Custom standards	BATCH_SIZE, WMASK, MASK bitmask constants named and documented for clarity.

10. Unit Test Strategy

All 12 tests pass using a zero-dependency micro-framework (static self-registration, no external libraries). Tests cover functional correctness, mathematical accuracy, throughput constraints, and full end-to-end pipeline validation.

Test Name	Category	Verifies
RingBuffer_PushPop	RingBuffer	Push single item; pop returns value; empty after pop
RingBuffer_Full	RingBuffer	4th push into capacity-4 buffer returns false
RingBuffer_FIFO_Order	RingBuffer	10 items pushed then popped in exactly same order
Filter_KnownInput_AllOnes	Filter math	All-ones window dot-product equals sum of weights
Filter_Weights_SumToOne	Filter math	Weight sum within 1% of 1.0 (energy-preserving)
Filter_KnownInput_DC	Filter math	DC input=127 produces output within 1 unit of 127
Threshold_AboveTV	Threshold	0.6 $\geq$ TV=0.5 $\rightarrow$ logical true
Threshold_BelowTV	Threshold	0.4 < TV=0.5 $\rightarrow$ logical false

Test Name	Category	Verifies
Threshold_ExactlyTV	Threshold	0.5 >= TV=0.5 → true (spec: >= is trigger)
DataGen_CsvLoad	CSV loading	2-row 4-col CSV produces correct PixelPair sequence
EndToEnd_KnownCsv	Integration	DC=200 through full pipeline with TV=100 → all outputs 1
Throughput_FilterUnder100ns	Throughput	10,000 filter calls average < 100 ns each

11. Project File Structure

File	Purpose
include/IProcessBlock.h	Pure-abstract base — configure/start/stop/join lifecycle
include/PixelPair.h	Data token: pixel1(u8) + pixel2(u8) + eos(bool), pack(1)
include/RingBuffer.h	Lock-free SPSC queue — bitmask, placement new, cache isolation, batch ops
include/PipelineConfig.h	All user parameters in one POD struct (m, TV, T, paths)
include/TimingProfiler.h	Header-only nanosecond profiler — min/mean/max/total
include/DataGenerationBlock.h	Camera emulator header — xorshift64, batch push, fast CSV parser
include/FilterThresholdBlock.h	Filter block header — Q24 weights, branchless threshold, batch pop
include/Pipeline.h	Orchestrator — vector of IProcessBlock, lifecycle management
include/CliParser.h	CLI argument parser — m, tv, t, csv, dump flags
src/DataGenerationBlock.cpp	xorshift64 PRNG, manual CSV byte-scan, batch push worker loop
src/FilterThresholdBlock.cpp	Q24 convolution, branchless threshold, merged sliding window
src/Pipeline.cpp	Block wiring, fork/join, profiling report
src/main.cpp	Entry point — signal handler, CSV dump, OutputCallback, display
tests/test_pipeline.cpp	12 self-registering unit tests, zero external dependencies
data/sample.csv	8×8 sample pixel array for test mode
Makefile	all / scanner / tests / run / run_csv / profile / clean targets

12. Build & Execution Instructions

12.1 Prerequisites

- GCC ≥ 7.0 or Clang ≥ 5.0 with C++17 support
- GNU Make
- Linux (primary) or Windows with WSL / MinGW

12.2 Build Commands

Command	Action
make	Build scanner binary + run all 12 unit tests

Command	Action
make scanner	Build scanner binary only
make tests	Build and execute unit tests
make run	Random-generator mode, 3-second timeout
make run_csv	CSV test mode with data/sample.csv
make profile	CSV mode at 500 μ s period — full timing report
make M=16 TV=0.3 T=500000 run_csv	Custom parameters in CSV mode
make clean	Remove build/ directory

12.3 Runtime Parameters

Flag	Type	Default	Description
--m	uint32	8	Number of columns (paper width)
--tv	double	0.5	Threshold value TV
--t	uint64	1000000	Cycle period T in nanoseconds (≥ 100)
--csv	path	(none)	CSV file path — enables test mode
--dump	path	(none)	Output CSV: index,raw,filtered,threshold

13. Design Questions Answered

Q1: Optimal architecture and design pattern?

Pipeline pattern with an **IProcessBlock** interface. Each block is a self-contained thread with a single responsibility (SRP). The interface decouples orchestration (Pipeline) from logic (blocks). Patterns used: Pipeline, RAII, Strategy (data source), Observer (output callback), Object Pool (ring buffer placement new), Null Object (default no-op callback).

Q2: Communication mechanism between blocks?

Lock-free SPSC ring buffer — RingBuffer<PixelPair, 256>. The single-producer / single-consumer guarantee is enforced by construction. Power-of-2 capacity enables bitmask wrapping (1 cycle). Placement new in raw byte storage eliminates all slot construction overhead. Cache-line-padded atomics eliminate false sharing. Batch ops amortise fence overhead by up to 64x.

Q3: Can the architecture SCALE with more blocks/features?

Yes, by design. Pipeline holds a `std::vector<IProcessBlock*>`. Adding Block 3 requires: (1) implement IProcessBlock in new .h/.cpp, (2) instantiate with a new RingBuffer channel, (3) append to Pipeline::build(). **Zero changes to any existing block, the interface, or the ring buffer.** The ring buffer type, PixelPair token, and OutputCallback can all be extended independently of each other.

Q4: Can the rest of the code be agnostic to changes — Modularity?

Yes. DataGenerationBlock has no knowledge of FilterThresholdBlock and vice versa — they share only the RingBuffer<PixelPair> type. Changing the filter kernel is a one-constant edit (FILTER_WINDOW / FILTER_Q24). Changing the data source is a runtime flag. Changing the threshold is a runtime parameter. Adding logging, telemetry, a third processing stage, or a different PRNG each touches exactly one file and leaves all others untouched.

End of Design Overview Document — CynLr Software Engineer Evaluation 1